

TRABAJO DE DOCUMENTACIÓN

Sistemas Informáticos 1ºDAM

David Cecilia, Sergio Lastras y Sergio Salido

Instalación, despliegue y funcionamiento de aplicaciones Java en entornos
reales

Índice

1) Introducción.....	3
2) Entorno de ejecución de aplicaciones java.....	4
2.1) Qué necesita un equipo para ejecutar Java.....	4
2.1.1) La Máquina Virtual Java (JVM).....	4
2.2) Diferencias entre JDK y JRE.....	4
2.3) Instalación y Verificación.....	5
2.3.1) Comprobación de la instalación.....	5
2.4) Variables de Entorno: JAVA_HOME y PATH.....	5
2.4.1) Problemas si no están configuradas correctamente.....	6
3) Creación de instaladores.....	6
3.1) Definición y ventajas del instalador.....	6
3.2) Herramientas para generar instaladores y ejecutables Java.....	7
3.3) Elementos que incluye un instalador.....	7
4) Servicios, APIs y comunicación en red.....	8
4.1) Qué es una API y su papel en el modelo cliente-servidor.....	8
4.1.2) En el modelo cliente-servidor.....	8
4.2) Spring Boot y su uso en servicios Java.....	9
4.3) Conceptos básicos de red relacionados con una API.....	9
5) Contenedores con Docker.....	9
5.1) Qué es Docker y qué problema resuelve.....	10
5.1.1) Diferencia entre máquina virtual y contenedor.....	10
5.2) Conceptos básicos de Docker.....	10
5.2.1) Relación de Docker con Java y Spring Boot.....	11
6) Aplicaciones frontend con Angular y React.....	11
6.1) Qué es una aplicación frontend y su papel en cliente-servidor.....	11
6.2) Angular y React.....	11
6.2.1) Ejecución: Node.js, npm y servidor de desarrollo.....	12
6.2.2) Build y archivos generados.....	12
6.2.3) Despliegue en servidor web.....	12
6.2.4) Relación con backend Java y APIs.....	12
7) Conclusión.....	13

1) Introducción

El desarrollo de software en Java ha evolucionado para convertirse en uno de los pilares de la informática moderna, gracias a su robustez y a su filosofía de portabilidad multiplataforma. Para que este principio se cumpla en entornos reales, no basta únicamente con escribir el código; es fundamental comprender el ecosistema tecnológico que permite su ejecución, despliegue y comunicación.

Este trabajo tiene como objetivo analizar los componentes esenciales para llevar una aplicación Java desde el entorno de desarrollo hasta el usuario final. En primer lugar, se explora el entorno de ejecución, detallando el papel crítico de la Máquina Virtual Java (JVM) y las diferencias entre el JRE (necesario para ejecutar aplicaciones) y el JDK (imprescindible para el desarrollo). Asimismo, se aborda la importancia de una correcta configuración del sistema mediante variables de entorno como `JAVA_HOME` y `PATH` para asegurar la operatividad de las herramientas.

Más allá de la ejecución básica, el documento profundiza en la creación de instaladores, una fase clave para dotar a las aplicaciones de un acabado profesional y facilitar su integración nativa en los sistemas operativos mediante herramientas como Launch4j o jpackage. Finalmente, se examina el papel de Java en el panorama actual de la comunicación en red, analizando el uso de APIs, el modelo cliente-servidor y el framework Spring Boot, estándar en la creación de servicios web y microservicios eficientes.

A través de estos puntos, se ofrece una visión integral sobre cómo se despliegan y operan las aplicaciones Java en entornos de producción reales.

2) Entorno de ejecución de aplicaciones java

Para que un ordenador sea capaz de entender y ejecutar el código escrito en Java, no basta con tener el archivo del programa. Java utiliza un ecosistema concreto que actúa como puente entre el código del programador y el hardware de la máquina.

2.1) Qué necesita un equipo para ejecutar Java

A diferencia de lenguajes como C++ que se compilan directamente a código máquina (binario específico para un procesador), Java se compila a un formato intermedio llamado Bytecode (archivos .class). Para ejecutar este Bytecode, el equipo necesita un entorno que lo interprete. Este entorno es el JRE (Java Runtime Environment).

2.1.1) La Máquina Virtual Java (JVM)

La JVM (Java Virtual Machine) es el corazón de la plataforma Java. Es un programa que simula un ordenador real dentro de tu sistema operativo. Su función principal es cargar el Bytecode, verificarlo y ejecutarlo.

Por qué permite la portabilidad (multiplataforma)

El lema de Java es "Write Once, Run Anywhere" (Escríbelo una vez, ejecútalo en cualquier lugar). Esto es posible porque:

1. El programador genera un único archivo Bytecode.
2. Existe una versión específica de la JVM para cada sistema operativo (Windows, Linux, macOS).
3. La JVM actúa como un traductor: toma el mismo Bytecode y lo traduce a las instrucciones específicas que cada sistema operativo entiende.

2.2) Diferencias entre JDK y JRE

Es común confundir estos términos, pero cumplen distintos roles dentro del ciclo de vida del software.

Componente	Definición	Uso Principal
JRE (Java Runtime Environment)	Contiene la JVM y las bibliotecas estándar de Java.	Ejecutar aplicaciones ya creadas. Es lo que necesita un usuario final.
JDK (Java Development Kit)	Contiene el JRE más herramientas de desarrollo (compilador javac, depuradores, etc.).	Desarrollar y crear aplicaciones. Es lo que necesita un programador.

2.3) Instalación y Verificación

Pasos básicos de instalación

1. Descarga: Acceder a la web de Oracle o utilizar distribuciones libres como OpenJDK (ej. Adoptium).
2. En Windows: Se descarga un instalador .msi o .exe. Se sigue el asistente de instalación asegurándose de anotar la ruta de instalación (habitualmente C:\Program Files\Java\jdk-xx).
3. En Linux (Ubuntu/Debian): Es más sencillo usar la terminal con el comando: `sudo apt update && sudo apt install default-jdk`.

2.3.1) Comprobación de la instalación

Una vez instalado, es vital verificar que el sistema reconoce los comandos de Java. Abre una terminal (CMD en Windows o Terminal en Linux) y escribe:

`java -version`: Muestra la versión instalada del entorno de ejecución.

`javac -version`: Muestra la versión del compilador (solo si instalaste el JDK).

Si la terminal devuelve la versión (ej. `java versión "21.0.1"`), la instalación básica es correcta.

2.4) Variables de Entorno: JAVA_HOME y PATH

Para que el sistema operativo encuentre los ejecutables de Java desde cualquier carpeta, es necesario configurar las variables de entorno.

1. JAVA_HOME

Es una variable que apunta a la ruta raíz donde se instaló Java (ej. `/usr/lib/jvm/java-11-openjdk` o `C:\Java\jdk-21`).

Sirve para que programas profesionales (como servidores web Tomcat o herramientas como Maven) busquen esta variable para saber dónde reside Java sin tener que preguntarle al usuario.

2. PATH

El PATH es una lista de directorios que el sistema operativo consulta cuando escribes un comando en la terminal.

Sirve para añadir la carpeta bin de Java (dentro de JAVA_HOME) al PATH. Así, al escribir `java` en la terminal, el sistema sabe que debe buscar ese ejecutable en la carpeta de instalación de Java sin necesidad de escribir la ruta completa.

2.4.1) Problemas si no están configuradas correctamente:

"Java no se reconoce como un comando interno o externo": Significa que el PATH no sabe dónde está el archivo java.exe.

Conflictos de versiones: Si tienes varias versiones de Java instaladas y el PATH no está bien ordenado, el sistema podría usar una versión antigua e incompatible.

Errores en Herramientas de Desarrollo: Los IDEs o herramientas de compilación pueden lanzar errores de "JAVA_HOME not set", impidiendo que el proyecto compile aunque Java esté físicamente

3) Creación de instaladores

Un instalador es un software diseñado específicamente para automatizar el proceso de colocar una aplicación en el sistema del usuario de forma que esté lista para ser utilizada. En el ecosistema Java, esto implica preparar no solo el código, sino también el entorno necesario para que funcione.

3.1) Definición y ventajas del instalador

Un instalador es un paquete que contiene todos los archivos necesarios de un programa y un script de instrucciones que guía al sistema operativo para su correcta integración.

Ventajas frente a archivos comprimidos (.zip) o .jar:

1. Profesionalidad y confianza: Un archivo .exe o .msi ofrece una imagen más formal que un simple archivo comprimido.
2. Automatización de dependencias: El instalador puede verificar si el JRE está presente y, si no, descargarlo o incluir una copia privada de Java.
3. Integración con el sistema: Crea entradas en el menú de inicio, accesos directos en el escritorio y registra la aplicación en el panel de control.
4. Proceso de desinstalación: Permite eliminar el software de forma limpia, borrando registros y carpetas que un archivo .zip dejaría huérfanos.
5. Permisos y rutas: El instalador gestiona los permisos de administrador necesarios para escribir en carpetas como Archivos de Programa.

3.2) Herramientas para generar instaladores y ejecutables Java

Para que una aplicación Java se sienta como una aplicación nativa, existen diversas herramientas que facilitan este proceso:

Launch4j: Es una herramienta que envuelve archivos .jar en ejecutables nativos de Windows (.exe). Permite configurar el icono de la aplicación y establecer una versión mínima de Java requerida.

Inno Setup: Es un instalador gratuito y de código abierto para Windows. Aunque no es exclusivo de Java, es muy utilizado para crear el paquete final que contiene el ejecutable generado previamente por otras herramientas.

package: Es la herramienta oficial incluida en el JDK (desde la versión 14). Permite empaquetar aplicaciones Java junto con una imagen reducida del JRE (utilizando jlink), generando instaladores nativos (.msi o .exe en Windows, .deb o .rpm en Linux).

install4j: Es una potente herramienta comercial que permite crear instaladores multiplataforma muy sofisticados, con interfaces gráficas personalizadas y gestión avanzada de actualizaciones.

3.3) Elementos que incluye un instalador

Un instalador completo suele estar compuesto por los siguientes módulos:

Archivos de la aplicación: El código compilado (.class o .jar), además de recursos necesarios como imágenes, sonidos o bases de datos locales.

Rutas de instalación: El proceso permite al usuario elegir dónde guardar el programa, aunque por defecto suele proponer carpetas estándar del sistema.

Accesos directos: Creación de iconos en el escritorio, en el menú de inicio o en la barra de tareas para facilitar el acceso al usuario.

Dependencias: Si la aplicación requiere bibliotecas externas o una versión específica de la JVM, el instalador puede incluirlas o asegurar su descarga.

Configuración inicial: Modificación de variables de entorno, creación de claves en el registro del sistema o archivos de configuración inicial (.ini o .properties).

Desinstalador: Un ejecutable adicional que se encarga de revertir todos los cambios realizados, asegurando que el equipo quede limpio si el usuario decide eliminar la aplicación.

4) Servicios, APIs y comunicación en red

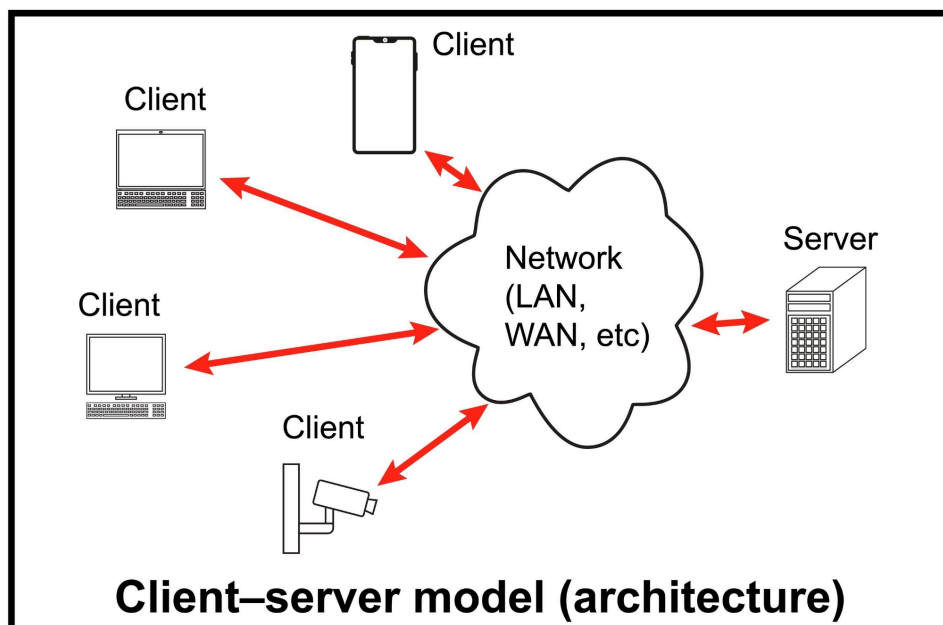
En el desarrollo de software moderno, las aplicaciones rara vez funcionan de forma aislada. Necesitan comunicarse con otros sistemas para intercambiar datos, y es aquí donde entran en juego las APIs y los servicios web.

4.1) Qué es una API y su papel en el modelo cliente-servidor

Una API (Application Programming Interface) es un conjunto de reglas y definiciones que permite que dos aplicaciones se comuniquen entre sí. Actúa como un contrato: la aplicación A (cliente) solicita algo siguiendo las reglas de la API, y la aplicación B (servidor) entrega una respuesta.

4.1.2) En el modelo cliente-servidor:

1. El Cliente es la aplicación que solicita el servicio (por ejemplo, una aplicación móvil o un navegador web).
2. El Servidor es el sistema que contiene los datos o la lógica de negocio.
3. La API es el intermediario. El cliente no necesita saber cómo funciona el servidor por dentro; solo necesita saber qué "botones" pulsar (puntos de acceso de la API) para obtener lo que necesita.



4.2) Spring Boot y su uso en servicios Java

Spring Boot es una herramienta (framework) diseñada para simplificar el desarrollo de aplicaciones basadas en Java. Se ha convertido en el estándar para crear microservicios y APIs REST.

Por qué se utiliza:

1. Configuración automática: Elimina la necesidad de configurar manualmente archivos XML complejos. El framework "adivina" qué componentes necesitas según las librerías que añadas.
2. Servidor embebido: Incluye un servidor web (como Tomcat) dentro de la propia aplicación. Esto significa que puedes ejecutar tu API simplemente ejecutando un archivo Java, sin instalar software de servidor externo.
3. Rapidez: Permite crear un servicio web funcional en cuestión de minutos con muy pocas líneas de código.

4.3) Conceptos básicos de red relacionados con una API

Para que la comunicación entre el cliente y la API tenga éxito, se deben manejar varios elementos técnicos:

Dirección IP: Es el identificador numérico único de un dispositivo en una red. Sirve para localizar físicamente dónde reside el servidor que aloja la API.

Puerto: Es un punto específico de conexión en la dirección IP. Si la IP es la dirección de un edificio, el puerto es el número de la oficina. Las APIs se suelen escuchar en puertos como el 80 (HTTP) o 443 (HTTPS), aunque en desarrollo es común usar el 8080.

Protocolo HTTP/HTTPS: Es el "idioma" que utilizan las aplicaciones para hablar. HTTPS es la versión segura que cifra los datos para que no puedan ser interceptados.

Peticiones GET y POST:

1. GET: Se utiliza cuando el cliente quiere pedir o "leer" información del servidor (por ejemplo, obtener una lista de productos).
2. POST: Se utiliza cuando el cliente quiere enviar datos nuevos al servidor para que sean procesados o guardados (por ejemplo, registrar un nuevo usuario).

Respuesta del servidor: Tras procesar la petición, el servidor devuelve un código de estado (como el famoso 200 OK si todo fue bien, o 404 Not Found si no existe lo solicitado) y, normalmente, un cuerpo de mensaje con la información en formato JSON, que es un formato de texto ligero y fácil de leer para las máquinas.

5) Contenedores con Docker

Docker ha transformado la manera en que se despliegan aplicaciones Java, eliminando los conflictos de configuración entre los equipos de los desarrolladores y los servidores de producción.

5.1) Qué es Docker y qué problema resuelve

Docker es una plataforma que permite empaquetar una aplicación y todas sus dependencias (librerías, sistema operativo ligero, configuración) en una unidad llamada contenedor.

El problema que resuelve es el clásico "en mi equipo funciona, pero en el servidor no". Antes de Docker, una aplicación Java podía fallar al desplegarse porque el servidor tenía una versión distinta de la JVM o faltaba una librería del sistema. Docker garantiza que la aplicación se ejecute en un entorno aislado y reproducible: si funciona en el contenedor de un desarrollador, funcionará exactamente igual en cualquier otro lugar.

5.1.1) Diferencia entre máquina virtual y contenedor

Aunque ambos sirven para aislar aplicaciones, su arquitectura es distinta.

Característica	Máquina Virtual (VM)	Contenedor (Docker)
Sistema Operativo	Incluye un SO completo (varios GB).	Comparte el núcleo del SO anfitrión (pocos MB).
Rendimiento	Más pesado y lento de arrancar.	Muy ligero y arranca en segundos.
Aislamiento	Total (más segura).	Aislamiento a nivel de procesos (muy eficiente).
Recursos	Consume mucha RAM y CPU de forma fija.	Consume solo lo que la aplicación necesita.

5.2) Conceptos básicos de Docker

Imagen: Es una plantilla de "solo lectura" que contiene el código, las herramientas y las librerías necesarias. Es como el instalador de un programa.

Contenedor: Es la instancia ejecutable de una imagen. Si la imagen es el plano de una casa, el contenedor es la casa ya construida. Puedes crear muchos contenedores a partir de una sola imagen.

Dockerfile: Un archivo de texto con instrucciones sencillas que Docker lee para construir una imagen automáticamente (ej. "Instala Java 21", "Copia mi archivo .jar", "Ejecuta este comando").

Volumen: Un mecanismo para persistir datos. Como los contenedores son temporales, si un contenedor se borra, sus datos mueren con él. Los volúmenes permiten guardar datos (como una base de datos) fuera del contenedor.

Puerto expuesto: Es la apertura que permite comunicar el mundo exterior con la aplicación dentro del contenedor (ej. mapear el puerto 8080 del PC al puerto 8080 del contenedor).

5.2.1) Relación de Docker con Java y Spring Boot

Para una aplicación Spring Boot, Docker es el compañero ideal. El proceso para empaquetarla es el siguiente:

1. Se genera el archivo .jar de la aplicación Spring Boot.
2. Se crea un Dockerfile que parte de una imagen oficial de Java (ej. OpenJDK).
3. En el Dockerfile se indica que se copie el .jar dentro de la imagen.
4. Se define el comando de inicio: `java -jar app.jar`.

Una vez creada la imagen, esta puede subirse a un repositorio (como Docker Hub). Cualquier otro equipo, sin necesidad de tener Java instalado ni configurar variables de entorno, puede descargar esa imagen y ejecutarla con un solo comando. La aplicación llevará su propia JVM y configuración "en la mochila", asegurando que se ejecute siempre de forma idéntica.

6) Aplicaciones frontend con Angular y React

En el desarrollo web actual, la interfaz que ve el usuario y la lógica de datos suelen estar separadas. Las aplicaciones frontend se encargan de todo lo relacionado con la experiencia visual y la interacción.

6.1) Qué es una aplicación frontend y su papel en cliente-servidor

Una aplicación frontend es la parte de un sistema con la que el usuario interactúa directamente a través de un navegador web. En el modelo cliente-servidor:

1. El Cliente (Frontend) se encarga de pintar la interfaz, capturar lo que el usuario escribe y mostrar los datos de forma atractiva.
2. El Servidor (Backend) gestiona la base de datos y la seguridad. La aplicación frontend se descarga en el navegador del usuario y se ejecuta allí, solicitando datos al backend solo cuando es necesario.

6.2) Angular y React

Son las dos herramientas más populares para construir frontends modernos, aunque tienen filosofías distintas:

Angular: Es un framework completo creado por Google. Incluye de serie todo lo necesario para crear aplicaciones grandes (gestión de formularios, rutas, servicios). Utiliza TypeScript, lo que lo hace muy estructurado y similar a la forma de programar en Java.

React: Es una biblioteca creada por Facebook enfocada en la creación de componentes visuales. Es más flexible y ligero que Angular, permitiendo a los desarrolladores elegir qué otras piezas quieren usar para las rutas o el manejo de datos.

6.2.1) Ejecución: Node.js, npm y servidor de desarrollo

Para desarrollar estas aplicaciones, se utilizan herramientas que se ejecutan fuera del navegador:

Node.js: Es el entorno que permite ejecutar JavaScript en el ordenador del desarrollador. Es necesario para que funcionen las herramientas de construcción.

npm (Node Package Manager): Es el gestor de paquetes. Se usa para descargar e instalar las librerías de Angular o React y cualquier otra herramienta que el proyecto necesite.

Servidor de desarrollo (dev server): Mientras se programa, se ejecuta un servidor local (normalmente mediante comandos como `npm start` o `ng serve`). Este servidor permite ver los cambios en tiempo real en el navegador cada vez que guardas un archivo, facilitando mucho el trabajo.

6.2.2) Build y archivos generados

El código que escribimos en Angular o React (con TypeScript o archivos modernos de JavaScript) no puede ser leído directamente de forma eficiente por todos los navegadores. Por eso, existe el proceso de build:

El build toma todo el código fuente y lo transforma, lo comprime y lo optimiza. El resultado es una carpeta (normalmente llamada `dist` o `build`) que contiene archivos estándar:

1. Archivos `.html` sencillos.
2. Archivos `.js` (JavaScript) que contienen toda la lógica.
3. Archivos `.css` para el diseño. Estos archivos son estáticos y son los que finalmente se envían al servidor.

6.2.3) Despliegue en servidor web

Una vez generada la carpeta del build, la aplicación frontend no necesita Node.js para funcionar en producción. Solo necesita un servidor web que sea capaz de servir archivos estáticos. Se pueden usar servidores como Nginx, Apache, o incluso servicios en la nube. El navegador del usuario pide el archivo `index.html`, y este se encarga de cargar el resto del JavaScript para que la aplicación empiece a funcionar.

6.2.4) Relación con backend Java y APIs

La aplicación frontend (Angular/React) y el backend (Java/Spring Boot) suelen vivir en lugares separados pero se mantienen en contacto constante:

1. El frontend envía peticiones HTTP (GET, POST) a la API creada en Java.
2. El backend procesa la petición y responde con datos, normalmente en formato JSON.
3. El frontend recibe ese JSON y lo "dibuja" en la pantalla para el usuario.

Esta separación permite que, por ejemplo, puedas cambiar todo el diseño de tu web (frontend) sin tener que tocar una sola línea de código de tu lógica de negocio en Java (backend).

7) Conclusión

El desarrollo en Java va mucho más allá de la escritura de código; su éxito radica en la robustez de su ecosistema. La portabilidad que ofrece la JVM es su mayor ventaja competitiva, pero esta debe estar respaldada por una configuración técnica impecable (manejo de JDK/JRE y variables de entorno) para garantizar la estabilidad del sistema.

Asimismo, la fase de despliegue mediante la creación de instaladores nativos es fundamental para profesionalizar el software, facilitando su adopción por parte del usuario final.

Finalmente, la integración con frameworks modernos como Spring Boot asegura que Java siga siendo la herramienta líder para crear servicios web eficientes y escalables en entornos de producción reales.